

FPGA 可视化编程工具 后端测试使用手册

版本 0.1.0 | 2026年5月

后端框架: FastAPI + Uvicorn

开发语言: Python 3.11+

模板引擎: Jinja2

数据模型: Pydantic v2

测试框架: pytest + httpx

本文档面向后端开发者和测试人员，详细说明每个API的功能、使用方法、测试步骤及预期结果。

目 录

- 1 项目概述
 - 1.1 项目背景与目标
 - 1.2 后端核心能力
 - 1.3 技术栈总览
- 2 环境准备
 - 2.1 Python 环境安装
 - 2.2 项目依赖安装
 - 2.3 验证安装
- 3 启动服务
 - 3.1 开发模式启动（推荐）
 - 3.2 命令行启动
 - 3.3 验证服务运行
- 4 项目架构详解
 - 4.1 目录结构
 - 4.2 数据流架构
 - 4.3 模块职责说明
- 5 IR 中间表示数据模型
 - 5.1 Port — 端口模型
 - 5.2 Module — 模块模型
 - 5.3 Connection — 连线模型
 - 5.4 WrappedModule — 封装模块
 - 5.5 IR — 顶层中间表示
 - 5.6 ProjectFile — 工程文件
 - 5.7 SimulationConfig — 仿真配置
 - 5.8 BoardConfig — 板卡配置
- 6 API 接口详细参考
 - 6.1 GET /api/health — 健康检查
 - 6.2 POST /api/generate/top — 生成顶层模块
 - 6.3 POST /api/generate/testbench — 生成Testbench
 - 6.4 POST /api/parse/verilog — 解析Verilog文件
 - 6.5 POST /api/parse/vcd — 解析VCD波形
 - 6.6 POST /api/project/save — 保存工程
 - 6.7 POST /api/project/load — 加载工程
 - 6.8 POST /api/simulate/compile — 编译HDL源文件
 - 6.9 POST /api/simulate/run — 运行仿真

7 测试指南

7.1 测试框架概述

7.2 运行全部测试

7.3 test_ir.py — 数据模型测试 (9个)

7.4 test_generators.py — 代码生成测试 (5个)

7.5 test_vcd.py — VCD解析测试 (4个)

7.6 test_api.py — API端点测试 (5个)

7.7 如何编写新测试

8 完整工作流演示

8.1 准备工作

8.2 生成顶层模块 — 详细步骤

8.3 生成Testbench — 详细步骤

8.4 解析Verilog文件

8.5 解析VCD波形

8.6 保存和加载工程

9 常见问题与故障排除

10 附录

10.1 依赖列表

10.2 demo_input.json 示例文件

10.3 ModelSim 环境要求

1. 项目概述

1.1 项目背景与目标

FPGA 可视化编程工具旨在为 FPGA 开发者提供一套类似于 ComfyUI 的节点式编程环境。用户可以通过拖拽模块节点、连接端口的方式完成 FPGA 设计，无需手写繁琐的顶层例化代码。工具最终可一键生成可综合的 Verilog/VHDL 顶层设计文件，并支持仿真联动与时序调试。

后端服务是整个工具的「引擎」，负责：

- (1) 接收前端画布生成的 IR（中间表示）JSON 数据
- (2) 根据 IR 生成 Verilog 顶层模块代码
- (3) 根据 IR 和仿真配置生成 Testbench 测试框架
- (4) 解析已有的 Verilog 文件，提取模块信息（用于工程导入）
- (5) 解析仿真产生的 VCD 波形文件（用于波形可视化）
- (6) 管理工程文件的保存与加载
- (7) 通过 subprocess 调用 ModelSim/Questa 进行仿真

1.2 后端核心能力

- > IR 驱动的 Verilog 顶层模块自动生成：自动推断顶层端口、创建 wire 声明、生成模块例化代码
- > Testbench 框架自动生成：时钟生成、复位序列、DUT 例化、激励占位区域、波形导出语句
- > Verilog 文件解析：基于正则表达式的解析器，支持 .v / .sv 文件，提取模块声明和例化关系
- > VCD 波形文件解析：自研解析器，支持 \$timescale、\$var、\$dumpvars、#time 标记和多 bit 总线值
- > ModelSim/Questa 仿真集成：通过 subprocess 异步调用 vlib/vlog/vsim，支持 TCL 脚本批处理
- > 工程文件管理：.fpga.json 格式的保存和加载，包含版本化管理
- > 完整的测试套件：23 个测试覆盖数据模型、代码生成、VCD 解析、API 端点

1.3 技术栈总览

后端使用的技术栈：

- > Web 框架：FastAPI — 基于 Starlette/Pydantic，异步支持，自动生成 OpenAPI 文档
- > ASGI 服务器：Uvicorn — 支持热重载，高性能异步处理
- > 数据校验：Pydantic v2 — 类型安全，自动 JSON Schema 生成
- > 模板引擎：Jinja2 — Verilog 代码模板渲染
- > 测试框架：pytest + pytest-asyncio + httpx — 异步 HTTP 测试
- > 仿真接口：subprocess + TCL 脚本 — 调用 ModelSim/Questa 命令行
- > 波形解析：自研 VCD 解析器 — 无外部依赖，支持基本 VCD 语法

2. 环境准备

2.1 Python 环境安装

后端服务需要 Python 3.11 或更高版本。请按以下步骤操作：

第1步 检查是否已安装 Python：

```
python --version
# 预期输出: Python 3.11.x 或更高版本
```

如果未安装 Python，请前往 <https://www.python.org/downloads/> 下载安装。安装时务必勾选「Add Python to PATH」选项。

第2步 检查 pip（Python 包管理器）是否可用：

```
pip --version
# 预期输出: pip 24.x 或类似版本信息
```

第3步 建议使用虚拟环境（可选但推荐）：

```
cd backend
python -m venv venv

# Windows 激活虚拟环境:
venv\Scripts\activate

# 激活成功后，命令行前面会显示 (venv) 前缀
```

第4步 确认 Python 版本满足要求 (≥ 3.11)：

```
python -c "import sys; assert sys.version_info >= (3, 11), 'Need Python 3.11+'; print('OK!')"
# 预期输出: OK
```

2.2 项目依赖安装

进入 backend 目录，使用 pip 安装所有依赖：

```
cd backend
pip install -r requirements.txt
```

此命令将安装以下核心依赖：

fastapi $\geq 0.111.0$	— Web 框架
uvicorn $\geq 0.29.0$	— ASGI 服务器
pydantic $\geq 2.7.0$	— 数据校验
jinja2 $\geq 3.1.4$	— 模板引擎
pytest $\geq 8.2.0$	— 测试框架
pytest-asyncio $\geq 0.23.7$	— 异步测试支持
httpx $\geq 0.27.0$	— HTTP 客户端（测试用）
fpdf $\geq 1.7.0$	— PDF 生成（手册生成用）

安装过程通常在 1-2 分钟内完成。如果下载速度较慢，可以使用国内镜像：

```
pip install -r requirements.txt -i https://pypi.tuna.tsinghua.edu.cn/simple
```

2.3 验证安装

安装完成后，执行以下命令验证所有依赖是否正确安装：

```
cd backend
python -c "import fastapi; import uvicorn; import pydantic; import jinja2; print('All OK!)"

# 预期输出: All OK
```

如果没有任何错误输出，说明环境已准备就绪。

还可以通过运行测试套件来全面验证：

```
cd backend
python -m pytest tests/ -v
# 预期输出: 23 passed
```

3. 启动服务

3.1 开发模式启动（推荐）

使用项目自带的 `main.py` 启动服务（已内置 `uvicorn` 配置）：

```
cd backend
python main.py
```

此命令将以开发模式启动服务，默认监听 `http://localhost:8000`。

服务启动后，你将在终端看到类似以下的输出：

```
INFO: Will watch for changes in these directories: ['D:\\...\\backend']
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO: Started reloader process
INFO: Started server process
INFO: Application startup complete.
```

看到「`Application startup complete.`」表示服务已成功启动。

3.2 命令行启动

也可以直接使用 `uvicorn` 命令启动，支持更多自定义参数：

```
cd backend
uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

常用参数说明：

- > `--reload`：开启热重载，代码修改后自动重启（开发必备）
- > `--host 0.0.0.0`：监听所有网络接口（如需局域网内其他设备访问）
- > `--port 8000`：指定端口号，可改为任意未占用端口
- > `--workers 4`：指定工作进程数（生产环境，不与 `--reload` 同时使用）

注意：如果端口 8000 被占用，可以换用其他端口如 `--port 8001`。不要同时启动多个实例监听同一端口。

3.3 验证服务运行

服务启动后，可以通过以下三种方式验证：

方式1 浏览器访问 `http://localhost:8000/api/health`，应看到 JSON 响应：

```
{"status": "ok", "service": "fpga-visual-tool-backend"}
```

方式2 使用 `curl` 命令测试：

```
curl http://localhost:8000/api/health
```

方式3 打开 Swagger UI 交互式文档（推荐用于手动测试和探索 API）：

```
# 浏览器打开：
http://localhost:8000/docs
```

Swagger UI 页面列出了所有可用的 API 端点。你可以直接在页面上填写参数、发送请求并查看响应结果。这是最方便的手动测试方式。

另外，访问 `http://localhost:8000/redoc` 可以查看 ReDoc 风格的文档。

4. 项目架构详解

4.1 目录结构

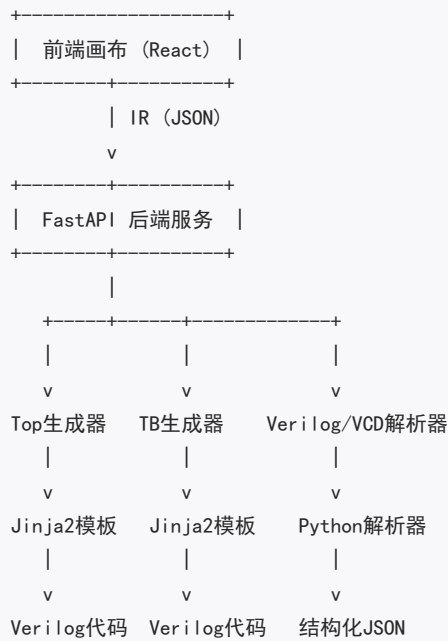
以下是 backend 目录的完整结构及各文件职责说明：

```
backend/
  main.py                # FastAPI 应用入口，创建 app 实例
  requirements.txt       # Python 依赖清单
  pyproject.toml         # pytest 测试配置
  demo_input.json        # 示例 IR 输入文件（用于快速测试）
  generate_manual.py      # PDF 手册生成脚本（英文版）
  generate_manual_cn.py  # PDF 手册生成脚本（中文版）
  app/
    __init__.py          # 包初始化
    routes.py            # 所有 API 路由定义（9个端点）
    models/
      __init__.py
      ir.py              # IR 数据模型定义（Pydantic v2）
    generators/
      __init__.py
      top.py             # 顶层模块 Verilog 代码生成器
      testbench.py       # Testbench 代码生成器
      templates/
        top.v.j2         # 顶层模块 Jinja2 模板
        testbench.v.j2   # Testbench Jinja2 模板
    parser/
      __init__.py
      verilog_parser.py  # Verilog 解析器（正则匹配）
    simulator/
      __init__.py
      base.py            # 仿真器抽象基类（接口定义）
      modelsim.py        # ModelSim/Questa 仿真器实现
    utils/
      __init__.py
      vcd_parser.py      # VCD 波形解析器（自研）
    tests/
      __init__.py
      test_ir.py          # IR 数据模型测试（9个）
      test_generators.py  # 代码生成器测试（5个）
      test_vcd.py         # VCD 解析器测试（4个）
      test_api.py         # API 端点集成测试（5个）
```

4.2 数据流架构

整个后端的数据流是单向的：前端画布 -> IR（JSON）-> 后端处理 -> 生成代码/解析结果。

IR（Intermediate Representation）是前后端之间的唯一交换格式，保证前后端解耦。



4.3 模块职责说明

`routes.py` (`app/routes.py`) — API 路由层:

- > 定义 9 个 API 端点的请求/响应模型 (Pydantic)
- > 使用单例模式管理生成器和解析器实例, 避免重复初始化
- > 统一的异常处理: 业务异常返回 HTTP 4xx, 系统异常返回 HTTP 5xx

`models/ir.py` (`app/models/ir.py`) — 数据模型层:

- > 定义 8 个核心 Pydantic 模型
- > 自动生成唯一 ID (uuid4 前12位十六进制)
- > IR 版本化 (version 字段), 支持向前/向后兼容

`generators/` — 代码生成层:

- > `top.py`: 扁平化模块和连线 -> 推断顶层端口 -> 构建 wire 声明 -> 生成例化语句 -> Jinja2 渲染
- > `testbench.py`: 提取仿真信号 -> 构建时钟/复位逻辑 -> 生成 DUT 例化 -> 添加 dump 语句 -> Jinja2 渲染
- > Jinja2 模板位于 `templates/` 目录, 模板与逻辑分离, 方便维护

`parser/verilog_parser.py` — 解析层:

- > 正则匹配 module 声明、端口定义、例化语句
- > 自动去除注释 (单行 `//` 和块注释 `/* */`)
- > 支持 `.v` / `.sv` 单文件解析和目录递归解析
- > VHDL 解析为占位实现, 生产环境需使用 `pyVHDLParser` 或 `GHDL`

`simulator/` — 仿真层:

- > `base.py`: 定义 Simulator 抽象基类, 声明 `compile/run/compile_and_run` 统一接口
- > `modelsim.py`: 通过 `asyncio.create_subprocess_exec` 异步调用 `ModelSim`
- > 自动生成 TCL 批处理脚本, 包含 `vcd add -r /*` 和 `run` 命令
- > 预留 Icarus Verilog / Verilator 实现扩展点

`utils/vcd_parser.py` — 工具层:

- > 自研 VCD 解析器，零外部依赖
- > 支持 \$timescale（时间单位）、\$var（信号定义）、\$dumpvars（初始值）、#time（时间标记）
- > 正确处理单 bit 值（0/1/x/z）和多 bit 总线值（b1010 格式）
- > 解析结果通过 to_dict() 方法输出为前端可直接消费的 JSON 结构

5. IR 中间表示数据模型

IR (Intermediate Representation) 是前端图形画布与后端代码生成之间的唯一数据交换格式。所有模型使用 Pydantic v2 定义，位于 `app/models/ir.py`。IR 包含版本号字段 (`version`)，用于保证向前/向后兼容性。

5.1 Port — 端口模型

描述模块的一个输入/输出/双向端口。

```
class PortDirection(str, Enum):
    INPUT = "input"      # 输入端口
    OUTPUT = "output"    # 输出端口
    INOUT = "inout"      # 双向端口

class Port(BaseModel):
    id: str               # 自动生成唯一ID (12位十六进制)
    name: str             # 端口名称, 如 clk, rst_n, data_in
    direction: PortDirection # 端口方向
    width: int = 1        # 位宽, 默认1
    signed: bool = False  # 有无符号
    array_size: int | None # 数组维度
    description: str = '' # 端口注释/说明
```

JSON 示例:

```
{
  "name": "clk",
  "direction": "input",
  "width": 1,
  "signed": false
}
```

5.2 Module — 模块模型

描述画布上的一个模块节点。

```
class Module(BaseModel):
    id: str               # 自动生成唯一ID
    name: str             # 模块名 (对应 Verilog module 名)
    instance_name: str | None # 用户自定义例化名
    type: str = "base"    # 模块类型: base/wrapped/board_ip
    ports: list[Port]     # 端口列表
    position: tuple[float, float] # 画布坐标 (x, y)
    config: dict = {}     # 模块参数配置, 如 {div_factor: 4}
```

模块类型说明:

- > base — 标准用户定义或从已有工程导入的模块
- > wrapped — 封装后的子电路模块, 可在画布上展开/折叠
- > board_ip — 板卡厂商提供的 IP 核 (如 PLL、GPIO、UART 等)

5.3 Connection — 连线模型

描述两个端口之间的连线关系，每条连线对应一根 wire。

```
class Connection(BaseModel):
    id: str          # 自动生成唯一ID
    src_module: str   # 源模块 ID
    src_port: str     # 源端口名称
    dst_module: str   # 目标模块 ID
    dst_port: str     # 目标端口名称
    wire_name: str | None # 用户标注的wire名, 为空则自动生成
```

5.4 WrappedModule — 封装模块

将一组内部模块打包为可复用的封装节点，支持嵌套封装。

```
class WrappedModule(BaseModel):
    name: str          # 封装后的模块名
    exposed_ports: list[Port] # 暴露到外部的端口
    internal_modules: list[Module] # 内部子模块
    internal_connections: list[Connection] # 内部连线
    internal_wrapped: list[WrappedModule] # 嵌套封装（递归）
```

5.5 IR — 顶层中间表示

画布完整状态的抽象，是前后端交换的核心对象。

```
class IR(BaseModel):
    version: str = "0.1.0" # IR 版本号
    modules: list[Module]   # 顶层模块列表
    connections: list[Connection] # 连线列表
    wrapped_modules: list[WrappedModule] # 封装模块
    top_module_name: str = "top" # 顶层模块名
```

5.6 ProjectFile — 工程文件

完整的工程存档格式（.fpga.json），包含 IR 及所有配置信息。

```
class ProjectFile(BaseModel):
    version: str = "0.1.0"
    name: str = "untitled"
    ir: IR          # IR 数据
    board: BoardConfig | None # 板卡配置（可选）
    simulation: SimulationConfig | None # 仿真配置（可选）
    canvas_state: dict # UI状态（视口位置、缩放等）
```

5.7 SimulationConfig — 仿真配置

```
class SimulationConfig(BaseModel):
    simulator: str = "modelsim" # 仿真器: modelsim/questa/icarus
    clock_period_ns: float = 10.0 # 时钟周期 (ns)
    reset_cycles: int = 10 # 复位持续周期数
    sim_time_us: float = 100.0 # 仿真总时长 (us)
    monitored_signals: list[str] # 待抓取信号列表
    # 格式: "module_id.port_name", 如 "m1.clk"
```

5.8 BoardConfig — 板卡配置

```
class BoardConfig(BaseModel):  
    board_name: str          # 板卡名称, 如 xilinx_zynq7000  
    fpga_part: str           # FPGA 型号  
    clock_pins: dict[str, str] # 时钟引脚映射  
    gpio_map: dict[str, str]  # GPIO 映射  
    constraints: str          # 约束文件内容 (.xdc / .qsf)
```

6. API 接口详细参考

所有 API 的基础 URL 为 `http://localhost:8000/api`。

请求和响应均使用 JSON 格式。

响应统一包含 "success" 字段（布尔值）表示操作是否成功。

测试 API 有三种方式：

- (1) 浏览器打开 `http://localhost:8000/docs` 使用 Swagger UI 交互式测试
- (2) 使用 `curl` 命令行测试（Windows PowerShell 亦可）
- (3) 使用 `pytest` 自动测试（推荐用于回归测试）

以下每个接口都提供了功能说明、请求/响应格式、`curl` 测试命令、PowerShell 测试命令和 `pytest` 测试命令。

6.1 GET /api/health — 健康检查

功能说明：检查后端服务是否正常运行。通常在服务启动后首先调用此接口确认服务可用。

请求格式：GET 请求，无需请求体。

响应格式：

```
HTTP 200 OK
Content-Type: application/json

{
  "status": "ok",
  "service": "fpga-visual-tool-backend"
}
```

`curl` 测试命令：

```
curl http://localhost:8000/api/health
```

PowerShell 测试命令：

```
Invoke-RestMethod -Uri http://localhost:8000/api/health | ConvertTo-Json
```

预期结果：返回 HTTP 200，响应体包含 `status="ok"`。

`pytest` 测试命令：

```
python -m pytest tests/test_api.py::test_health -v
```

测试代码片段（来自 `tests/test_api.py`）：

```
@pytest.mark.asyncio
async def test_health(client: AsyncClient):
    resp = await client.get("/api/health")
    assert resp.status_code == 200
    data = resp.json()
    assert data["status"] == "ok"
```

6.2 POST /api/generate/top — 生成顶层模块

功能说明：

这是最核心的 API。接收一个 IR 对象，自动生成完整的 Verilog 顶层模块代码。

生成器会自动完成以下工作：

- (1) 遍历所有模块和封装子模块，扁平化处理
- (2) 推断顶层端口：未与其他模块连接的端口自动暴露为顶层 I/O，按名称去重
- (3) 为所有连线生成 wire 声明（使用用户标注的 wire 名或自动生成 w_xxx 格式的名称）
- (4) 位宽自动匹配：取连线两端中较大的位宽
- (5) 为所有子模块生成例化语句，未连接的端口悬空处理
- (6) 通过 Jinja2 模板渲染输出最终的 Verilog 代码

请求格式：

```
POST /api/generate/top
Content-Type: application/json

{
  "ir": {
    "version": "0.1.0",
    "modules": [ ... ],
    "connections": [ ... ],
    "wrapped_modules": [],
    "top_module_name": "my_design"
  }
}
```

响应格式：

```
{
  "success": true,
  "verilog": "module my_design(...)\n  ...\nendmodule\n"
}
```

curl 测试命令（使用项目自带的 demo_input.json）：

```
curl -X POST http://localhost:8000/api/generate/top ^
-H "Content-Type: application/json" ^
-d @demo_input.json
```

PowerShell 测试命令：

```
$body = Get-Content demo_input.json -Raw
$resp = Invoke-RestMethod -Uri http://localhost:8000/api/generate/top -Method Post -Body $body -ContentType "application/json"
$resp.success
Write-Output $resp.verilog
```

预期结果：返回一段完整的 Verilog 代码，包含 module blinky_demo 声明、input/output 端口声明、wire clk_divided 声明、clk_divider 和 led_controller 的例化。

pytest 测试命令：

```
python -m pytest tests/test_api.py::test_generate_top -v
```

6.3 POST /api/generate/testbench — 生成 Testbench

功能说明：

根据 IR 和仿真配置自动生成 Testbench 框架代码。

生成的 Testbench 包含：

- (1) 时钟生成逻辑（周期通过 `clock_period_ns` 可配置）
- (2) 可配置的复位序列（持续 `reset_cycles` 个周期后释放）
- (3) DUT（待测模块）例化
- (4) 激励信号声明和占位区域（标记 `// STIMULUS`：供用户添加测试激励）
- (5) `$dumpfile / $dumpvars` 波形导出语句
- (6) 仿真超时后自动调用 `$finish` 结束仿真

请求格式：

```
POST /api/generate/testbench
Content-Type: application/json

{
  "ir": { IR对象 },
  "simulation": {
    "clock_period_ns": 10.0,
    "reset_cycles": 5,
    "sim_time_us": 100.0
  }
}
```

响应格式：

```
{
  "success": true,
  "verilog": "module tb_xxx...\nendmodule\n"
}
```

curl 测试命令：

```
curl -X POST http://localhost:8000/api/generate/testbench ^
-H "Content-Type: application/json" ^
-d '{"ir":{"version":"0.1.0","modules":[],"connections":[],"wrapped_modules":[],"top_module_name":"tb_blinky"},"simulation":{"clock_period_ns":10.0,"reset_cycles":5,"sim_time_us":100.0}}'
```

预期结果：返回 Testbench 代码，包含 `tb_blinky` 模块声明、`clk` 生成逻辑（`forever #5.0`）、`rst_n` 复位序列（`repeat(5)`）、`$dumpfile / $dumpvars` 语句和 `$finish` 语句。

pytest 测试命令：

```
python -m pytest tests/test_api.py::test_generate_testbench -v
```


6.4 POST /api/parse/verilog — 解析 Verilog 文件

功能说明：

解析指定的 Verilog 文件（或目录），提取其中定义的模块信息。

用于「导入已有工程」功能。支持 .v 和 .sv 扩展名。

解析结果包括：模块名称、端口列表（方向/位宽/有无符号）、内部子模块例化关系。

注意：当前为基于正则表达式的解析器（demo 质量），对于复杂的 SystemVerilog 语法（如 generate、interface、parameterized module）可能解析不完整。

请求格式：

```
POST /api/parse/verilog
Content-Type: application/json

{
  "file_path": "D:/projects/my_module.v"
}
```

响应格式：

```
{
  "success": true,
  "modules": [
    {
      "name": "counter",
      "ports": [
        {"name": "clk", "direction": "input", "width": 1},
        {"name": "q", "direction": "output", "width": 4}
      ],
      "instantiations": [...]
    }
  ]
}
```

PowerShell 测试命令：

```
# 1. 创建测试用 Verilog 文件
Set-Content -Path test_sample.v -Value @"
module counter(input clk, input rst_n, output reg [3:0] q);
  always @(posedge clk) q <= q + 1;
endmodule
"@

# 2. 调用解析 API
$body = {"file_path": "test_sample.v"}
$resp = Invoke-RestMethod -Uri http://localhost:8000/api/parse/verilog -Method Post -Body $body -ContentType "application/json"
$resp.success
$resp.modules | ConvertTo-Json
```

预期结果：返回 modules 数组，包含 counter 模块及其 2 个端口（clk 和 q）的信息。

如果文件不存在，返回 HTTP 404 错误。解析器对 generate、interface 等复杂语法支持有限。

6.5 POST /api/parse/vcd — 解析 VCD 波形文件

功能说明：

解析仿真产生的 VCD (Value Change Dump) 波形文件，返回结构化的波形数据供前端渲染波形图。

解析器支持：

\$timescale — 时间单位定义

\$var — 信号定义（名称、位宽、ID 代码）

\$dumpvars — 初始值

#time — 时间标记

单 bit 值 (0/1/x/z) 和多 bit 总线值 (b1010 格式)

请求格式：

```
POST /api/parse/vcd
Content-Type: application/json

{
  "file_path": "D:/sim_output/waveform.vcd"
}
```

响应格式：

```
{
  "success": true,
  "waveform": {
    "signals": [
      {
        "name": "clk",
        "width": 1,
        "signed": false,
        "changes": [
          {"time_ns": 0, "value": "0"},
          {"time_ns": 10, "value": "1"},
          {"time_ns": 20, "value": "0"}
        ]
      }
    ]
  },
  "total_time_ns": 20,
  "timescale": "1ns"
}
```

每个信号包含一个 `changes` 数组，记录了每次值变化的时间点和值。`total_time_ns` 为整个波形的最大时间戳。

PowerShell 测试命令：

```
# 1. 创建测试 VCD 文件
Set-Content -Path test_wave.vcd -Value @"
$timescale 1ns $end
$var wire 1 ! clk $end
$dumpvars
0!
$end
#10
1!
"@

# 2. 调用解析 API
$body = {"file_path": "test_wave.vcd"}
$resp = Invoke-RestMethod -Uri http://localhost:8000/api/parse/vcd -Method Post -Body $body -ContentType "application/json"
$resp.success
$resp.waveform.signals | ConvertTo-Json -Depth 5
```

预期结果：返回 clk 信号，包含 2 次值变化（0ns 时值为 0，10ns 时值为 1）。

pytest 测试命令：

```
python -m pytest tests/test_vcd.py -v
```

6.6 POST /api/project/save — 保存工程

功能说明：

将完整的工程数据（包括 IR、板卡配置、仿真配置、画布 UI 状态）保存为 .fpga.json 文件。

文件以 UTF-8 编码写入，JSON 格式化输出（indent=2），便于版本控制。

目标目录不存在时会自动创建。

请求格式：

```
POST /api/project/save
Content-Type: application/json

{
  "project": { ProjectFile 对象 },
  "file_path": "D:/projects/my_design.fpga.json"
}
```

响应格式：

```
{
  "success": true,
  "path": "D:/projects/my_design.fpga.json"
}
```

PowerShell 测试命令：

```
$proj = @{
  version = "0.1.0"
  name = "my_test_project"
  ir = @{
    version = "0.1.0"
    modules = @()
    connections = @()
    wrapped_modules = @()
    top_module_name = "top"
  }
}

$body = @{ project = $proj; file_path = "test_project.fpga.json" } | ConvertTo-Json -Depth 6
Invoke-RestMethod -Uri http://localhost:8000/api/project/save -Method Post -Body $body -ContentType "application/json"
```

预期结果：在当前目录生成 test_project.fpga.json 文件，包含完整的工程数据结构。

6.7 POST /api/project/load — 加载工程

功能说明：

从磁盘加载一个 .fpga.json 工程文件，返回完整的 ProjectFile 对象。

文件不存在时返回 HTTP 404 错误。

JSON 格式无效时返回 HTTP 400 错误。

加载时使用 Pydantic 的 model_validate 对数据进行完整校验。

请求格式：

```
POST /api/project/load
Content-Type: application/json

{
  "file_path": "test_project.fpga.json"
}
```

响应格式：

```
{
  "success": true,
  "project": {
    "version": "0.1.0",
    "name": "my_test_project",
    "ir": { ... }
  }
}
```

PowerShell 测试命令（先保存再加载）：

```
$body = {"file_path": "test_project.fpga.json"}
$resp = Invoke-RestMethod -Uri http://localhost:8000/api/project/load -Method Post -Body $body -ContentType "application/json"
$resp.success
$resp.project.name
```

预期结果：返回 success=true, project.name 为保存时设置的名称 my_test_project。

pytest 集成测试（保存+加载）：

```
python -m pytest tests/test_api.py::test_save_and_load_project -v
```

6.8 POST /api/simulate/compile — 编译 HDL 源文件

功能说明：

使用 ModelSim/Questa 的 vlib 和 vlog 命令编译 Verilog 源文件。

执行流程：(1) vlib work 创建 work 仿真库；(2) 对每个源文件执行 vlog -work work <file>。

编译过程中产生的错误（errors）和警告（warnings）会被捕获并返回给调用者。

work_dir 目录不存在时会自动创建。

此接口需要本地安装 ModelSim/Questa，且 vlib/vlog 命令必须在系统 PATH 中，或通过 ModelSimSimulator 的 executable_path 参数指定安装路径。

请求格式：

```
POST /api/simulate/compile
Content-Type: application/json

{
  "sources": ["src/counter.v", "src/top.v"],
  "work_dir": "./sim_work"
}
```

响应格式：

```
{
  "success": true,
  "stdout": "编译输出日志...",
  "stderr": "",
  "errors": [],
  "warnings": []
}
```

如果编译失败（success=false），检查 errors 数组获取详细错误信息。stdout/stderr 返回最后 2000 个字符。

6.9 POST /api/simulate/run — 运行仿真

功能说明：

使用 ModelSim/Questa 的 vsim 命令运行仿真。

执行流程：(1) 生成 TCL 批处理脚本（vcd file / vcd add -r /* / run <time> / vcd flush / quit）

(2) 执行 vsim -c -do <tcl_script> work.<top_module>

(3) 检查 VCD 文件是否生成，返回 VCD 文件路径

支持两种模式：直接运行（已有编译好的 work 库）或编译+运行（提供 sources 参数）。

此接口需要本地安装 ModelSim/Questa。确保 vsim 命令在系统 PATH 中可用。

请求格式：

```
POST /api/simulate/run
Content-Type: application/json

{
  "top_module": "my_top",
  "sim_time": "1us",
  "work_dir": "./sim_work",
  "sources": ["src/counter.v", "src/top.v"]
  // sources 为可选参数，提供后将先编译再运行
}
```

响应格式：

```
{
  "success": true,
  "vcd_path": "./sim_work/my_top.vcd",
  "stdout": "仿真输出日志...",
  "errors": [],
  "warnings": []
}
```

7. 测试指南

7.1 测试框架概述

项目包含 23 个测试用例，分布在 4 个测试文件中，全部通过。

测试框架：pytest + pytest-asyncio（异步测试）+ httpx（HTTP 客户端）

测试配置：pyproject.toml 中 asyncio_mode = auto，自动管理事件循环

API 测试特色：使用 httpx AsyncClient + ASGI transport，直接在内存中测试 app 实例，无需启动真实服务器，测试执行速度快。

7.2 运行全部测试

在 backend 目录下执行以下命令：

```
cd backend
python -m pytest tests/ -v
```

预期输出：23 passed in ~0.5s，全部绿色通过。

常用测试选项：

- > 查看详细输出（含 print 语句）：python -m pytest tests/ -v -s
- > 仅运行失败的测试：python -m pytest tests/ --lf
- > 先运行失败的再运行其他的：python -m pytest tests/ --ff
- > 生成覆盖率报告（需安装 pytest-cov）：pip install pytest-cov && python -m pytest tests/ --cov=app --cov-report=html
- > 指定单个测试文件：python -m pytest tests/test_ir.py -v

7.3 test_ir.py — 数据模型测试（9个）

测试内容：验证所有 Pydantic 数据模型的创建、属性默认值、序列化和反序列化。

```
python -m pytest tests/test_ir.py -v
```

包含 5 个测试类：

- > TestPort （3个）： test_create_simple_port — 验证默认参数（width=1, signed=False）； test_create_bus_port — 验证多位宽有符号端口； test_auto_id — 验证自动生成的 ID 长度为 12 且唯一
- > TestModule （2个）： test_create_module — 验证空模块创建（type=base, ports=[]）； test_module_with_ports — 验证带 2 个端口的模块
- > TestConnection （1个）： test_connection — 验证连线创建，含自定义 wire_name
- > TestIR （2个）： test_empty_ir — 验证默认 IR（version=0.1.0, name=top, 空列表）； test_ir_with_data — 验证完整 IR 创建
- > TestProjectFile （2个）： test_default_project — 验证默认工程； test_serialize_deserialize — 验证 model_dump() 和 model_validate() 的往返正确性

7.4 test_generators.py — 代码生成器测试（5个）

测试内容：验证 TopGenerator 和 TestbenchGenerator 生成的 Verilog 代码的正确性。

```
python -m pytest tests/test_generators.py -v
```


包含 2 个测试类：

- > TestTopGenerator (3个): test_generate_simple_top — 2模块1连线生成包含 module/endmodule/wire/例化; test_infer_top_ports — 未连接端口暴露为顶层 I/O; test_multiple_connections — 多连线场景 wire 名正确性
- > TestTestbenchGenerator (2个): test_generate_tb — 验证 TB 包含时钟/复位/DUT/\$dumpfile/\$finish; test_tb_includes_clock_period — 验证时钟周期值出现在输出中

7.5 test_vcd.py — VCD 解析器测试 (4个)

测试内容：验证自研 VCD 解析器对 VCD 文件内容的正确解析。

```
python -m pytest tests/test_vcd.py -v
```

包含 2 个测试类：

- > TestVCDParser (3个): test_parse_simple_vcd — 1bit+4bit 信号的完整 VCD; test_empty_vcd — 空 VCD 返回空信号列表
- > TestWaveformData (1个): test_to_dict — 验证 to_dict() 输出的 JSON 结构正确

7.6 test_api.py — API 端点集成测试 (5个)

测试内容：端到端 HTTP 测试，验证所有主要 API 端点。使用 ASGI transport 直接在内存中测试，无需启动服务器。

```
python -m pytest tests/test_api.py -v
```

- > test_health — GET /api/health -> 200 + status=ok
- > test_generate_top — POST /api/generate/top -> 验证生成的 Verilog 包含 module/endmodule
- > test_generate_testbench — POST /api/generate/testbench -> 验证 TB 包含模块名
- > test_save_and_load_project — 保存+加载往返测试 -> 验证加载的工程名一致
- > test_parse_vcd_not_found — 解析不存在的 VCD 文件 -> 验证返回 404

7.7 如何编写新测试

编写数据模型测试（参考 test_ir.py 的模式）：

```
# 在 tests/ 下创建 test_xxx.py
import pytest
from app.models.ir import Port, PortDirection

class TestYourFeature:
    def test_something(self):
        p = Port(name="test", direction=PortDirection.INPUT)
        assert p.name == "test"
        assert p.width == 1
```

编写 API 测试（参考 test_api.py 的模式）：

```
import pytest
from httpx import AsyncClient, ASGITransport
from main import app

@pytest.fixture
async def client():
    transport = ASGITransport(app=app)
    async with AsyncClient(transport=transport, base_url="http://test") as ac:
        yield ac

@pytest.mark.asyncio
async def test_my_endpoint(client: AsyncClient):
    resp = await client.post("/api/my/endpoint", json={"key": "value"})
    assert resp.status_code == 200
    assert resp.json()["success"]
```

8. 完整工作流演示

本章演示一个完整的端到端工作流：从启动服务到生成顶层模块、生成 Testbench、解析文件、保存/加载工程。使用项目自带的 `demo_input.json` 示例文件。

8.1 准备工作

第1步 打开终端，进入 `backend` 目录并启动服务：

```
cd D:\claude\prj\backend
python main.py
```

看到「Application startup complete.」说明服务已就绪。

第2步 打开另一个终端窗口（保留服务运行），验证服务：

```
curl http://localhost:8000/api/health
```

预期返回：{"status": "ok", "service": "fpga-visual-tool-backend"}

第3步 打开 Swagger UI（可选，方便可视化测试）：

```
# 浏览器打开: http://localhost:8000/docs
```

8.2 生成顶层模块 — 详细步骤

第1步 查看 `demo_input.json` 的内容。该文件包含 2 个模块（`clk_divider`, `led_controller`）和 1 根连线（`clk_out -> clk`），顶层名为 `blinky_demo`。

第2步 发送生成请求：

```
curl -X POST http://localhost:8000/api/generate/top ^
-H "Content-Type: application/json" ^
-d @demo_input.json
```

第3步 预期的生成结果（关键内容）：

```
module blinky_demo (
    input  clk_in,
    input  rst_n,
    output [3:0] led_out
);

wire clk_divided;

clk_divider u_div (
    .clk_in (clk_in),
    .rst_n  (rst_n),
    .clk_out(clk_divided)
);

led_controller u_led (
    .clk      (clk_divided),
    .rst_n    (rst_n),
    .led_out  (led_out)
);

endmodule
```

验证要点：

- > clk_in/rst_n 被推断为顶层 input（这些端口虽然被连接但未被其他模块的输出驱动）
- > led_out 被推断为顶层 output（未被任何模块的 input 连接）
- > clk_divided 被声明为内部 wire（连接 m1.clk_out 和 m2.clk）
- > 两个模块的例化端口映射正确

8.3 生成 Testbench — 详细步骤

使用与上一步相同的 IR 结构，加上仿真配置参数：

```
curl -X POST http://localhost:8000/api/generate/testbench ^
-H "Content-Type: application/json" ^
-d '{"ir\':"{"version\":"0.1.0\","modules\":[],"connections\":[],"wrapped_modules\":[],"top_module_name\":"blinky\"},"simu
```

预期输出包含以下关键内容：

- > module tb_blinky — Testbench 模块声明
- > forever #5.0 clk = ~clk — 10ns 周期的时钟生成
- > repeat(5) @(posedge clk) — 5 个周期的复位序列
- > blinky u_dut(...) — DUT 例化
- > \$dumpfile / \$dumpvars — 波形导出配置
- > #100000; \$finish — 100us 后自动结束仿真

8.4 解析 Verilog 文件

第1步 创建测试用 Verilog 文件：

```
Set-Content -Path test_sample.v -Value @"
module counter(input clk, input rst_n, output reg [3:0] q);
    always @(posedge clk) q <= q + 1;
endmodule
"@
```

第2步 调用解析 API：

```
$body = {"file_path": "test_sample.v"}
Invoke-RestMethod -Uri http://localhost:8000/api/parse/verilog -Method Post -Body $body -ContentType "application/json" | ConvertTo-
```

预期返回 counter 模块的端口信息：clk(input,1bit), rst_n(input,1bit), q(output,4bit)。

8.5 解析 VCD 波形

第1步 创建测试 VCD 文件：

```
Set-Content -Path test_wave.vcd -Value @"
$timescale 1ns $end
$var wire 1 ! clk $end
$dumpvars
0!
$end
#10
1!
"@
```

第2步 调用解析 API：

```
$body = {"file_path": "test_wave.vcd"}
Invoke-RestMethod -Uri http://localhost:8000/api/parse/vcd -Method Post -Body $body -ContentType "application/json" | ConvertTo-Json
```

预期返回 clk 信号的波形数据，包含 2 次值变化（0ns 时值为 0，10ns 时值为 1）。

8.6 保存和加载工程

第1步 保存工程：

```
$proj = @{version="0.1.0"; name="demo"; ir=@{version="0.1.0"; modules=@(); connections=@(); wrapped_modules=@(); top_module_name="top_module_name";  
$body = @{project=$proj; file_path="demo_project.fpga.json"} | ConvertTo-Json -Depth 6  
Invoke-RestMethod -Uri http://localhost:8000/api/project/save -Method Post -Body $body -ContentType "application/json"
```

第2步 加载工程：

```
$body = {"file_path": "demo_project.fpga.json"}  
$resp = Invoke-RestMethod -Uri http://localhost:8000/api/project/load -Method Post -Body $body -ContentType "application/json"  
$resp.project.name
```

预期加载的工程 name 为 demo，与保存时一致。

9. 常见问题与故障排除

Q1: 启动服务时提示端口被占用

错误信息: Address already in use

解决方法 A: 更换端口号启动。

```
uvicorn main:app --reload --host 0.0.0.0 --port 8001
```

解决方法 B: 查找并终止占用端口的进程。

```
# PowerShell 查找占用 8000 端口的进程:
netstat -ano | findstr :8000
# 记下最后一列的 PID, 然后终止:
taskkill /PID <PID> /F
```

Q2: 导入模块时报 ModuleNotFoundError

错误信息: ModuleNotFoundError: No module named 'app'

解决方法: 确保在 backend 目录下执行命令, pyproject.toml 中已配置 pythonpath = ["."]。

```
cd D:\claude\prj\backend
python -m pytest tests/ -v

# 或手动设置 PYTHONPATH:
$env:PYTHONPATH = "D:\claude\prj\backend"
python -m pytest tests/
```

Q3: pytest 找不到测试文件

确认 pyproject.toml 中已配置 testpaths 和 pythonpath:

```
[tool.pytest.ini_options]
testpaths = ["tests"]
pythonpath = ["."]
```

同时确认测试文件命名符合 pytest 默认规则 (test_*.py 或 *_test.py)。

Q4: 代码生成结果中端口丢失或 wire 名不对

可能原因:

- > IR 中 module.id 和 Connection 中的 src_module/dst_module 不匹配 — 检查 ID 引用
- > 端口名称大小写不一致 — 确认 IR 中的 port name 与实际 Verilog module 的端口名一致
- > 封装模块的内部连线未正确扁平化 — 检查 WrappedModule 的 internal_connections 是否正确传入

排查方法: 打印 IR JSON 内容, 逐字段与预期值比对。

Q5: 仿真接口报错 vlib/vlog/vsim not found

原因: ModelSim/Quarta 未安装或不在系统 PATH 环境变量中。

解决方法:

- > 安装 ModelSim/Quarta 到本地 (如 C:\modeltech64_10.7)
- > 将安装路径 (如 C:\modeltech64_10.7\win64) 添加到系统 PATH 环境变量
- > 或在代码中创建 ModelSimSimulator 时传入 executable_path 参数指定安装目录

Q6: VCD 解析结果为空或不完整

可能原因：

- > VCD 文件格式不规范 — 某些 EDA 工具生成的 VCD 有变体语法
- > \$var 定义使用了多行格式 — 解析器已支持多行，但极特殊情况可能遗漏
- > 信号 ID 代码使用了非标准字符 — 解析器对 ID 代码有字符限制

排查方法：先手动查看 VCD 文件内容，确认格式。然后用简单测试 VCD 验证解析器是否正常。

Q7: curl 命令在 PowerShell 中执行失败

Windows PowerShell 中的 curl 可能指向 Invoke-WebRequest 的别名。

解决方法：使用 curl.exe 明确调用，或直接使用 PowerShell 的 Invoke-RestMethod。

```
# 方法1: 使用 curl.exe
curl.exe http://localhost:8000/api/health

# 方法2: 使用 Invoke-RestMethod
Invoke-RestMethod -Uri http://localhost:8000/api/health
```

10. 附录

10.1 依赖列表

requirements.txt 完整内容:

```
fastapi      >= 0.111.0    # Web 框架
uvicorn      >= 0.29.0     # ASGI 服务器
pydantic     >= 2.7.0       # 数据校验
jinja2       >= 3.1.4     # 模板引擎
vcdvcd       >= 2.0.0      # VCD 库 (辅助, 非核心依赖)
pytest       >= 8.2.0     # 测试框架
pytest-asyncio >= 0.23.7  # 异步测试支持
httpx        >= 0.27.0    # HTTP 测试客户端
fpdf         >= 1.7.0     # PDF 生成 (手册生成用)
```

10.2 demo_input.json 示例文件

项目根目录下的 demo_input.json 包含了一个完整的 IR 示例, 可作为测试数据直接使用:

```
{
  "ir": {
    "version": "0.1.0",
    "modules": [
      {
        "id": "m1", "name": "clk_divider", "instance_name": "u_div", "type": "base",
        "ports": [
          {
            "name": "clk_in", "direction": "input", "width": 1,
            "name": "rst_n", "direction": "input", "width": 1,
            "name": "clk_out", "direction": "output", "width": 1
          ]
        },
        "position": [0, 0], "config": {
          "div_factor": 4
        },
        {
          "id": "m2", "name": "led_controller", "instance_name": "u_led", "type": "base",
          "ports": [
            {
              "name": "clk", "direction": "input", "width": 1,
              "name": "rst_n", "direction": "input", "width": 1,
              "name": "led_out", "direction": "output", "width": 4
            }
          ],
          "position": [300, 0], "config": {}
        }
      ],
      "connections": [
        {
          "id": "c1", "src_module": "m1", "src_port": "clk_out",
          "dst_module": "m2", "dst_port": "clk", "wire_name": "clk_divided"
        }
      ],
      "wrapped_modules": [],
      "top_module_name": "blinky_demo"
    }
  }
}
```

此示例描述了一个简单的设计: clk_divider 模块产生分频时钟 clk_divided, 输出给 led_controller 模块驱动 LED。该文件可直接用于测试 /api/generate/top 接口。

10.3 ModelSim 环境要求

仿真功能 (/api/simulate/compile 和 /api/simulate/run) 需要本地安装 ModelSim 或 Questa Sim。

安装后, 以下命令需要在终端中可直接调用:

- > vlib — 创建仿真库 (vlib work)
- > vlog — 编译 Verilog 源文件 (vlog -work work <source>)
- > vcom — 编译 VHDL 源文件 (如果使用 VHDL)
- > vsim — 运行仿真 (vsim -c -do <tcl_script>)

如果未安装 ModelSim, 仿真相关的 API 将返回错误 (success=false), 但其他所有 API 功能不受影响。

对于 CI/CD 环境或没有 ModelSim 许可证的情况, 可以考虑以下替代方案:

- (1) 使用免费的 Icarus Verilog (iverilog) 进行仿真验证
- (2) 使用 Verilator 进行 C++/SystemC 协同仿真
- (3) 实现对应的 Simulator 子类来集成其他仿真器

-- 手册结束 --